

תרגיל בית 2 - חלק יבש

מבנה הנתונים:

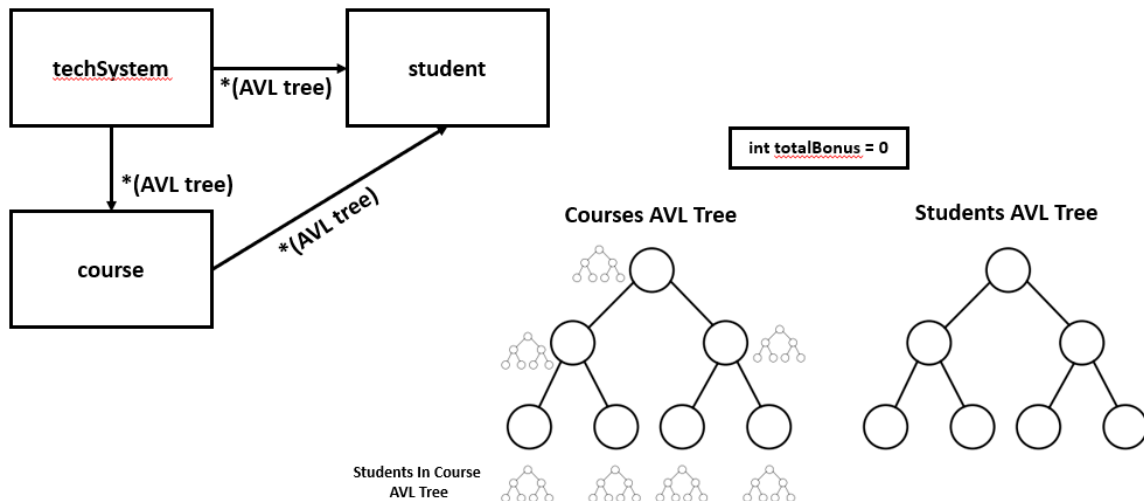
לטובת מימוש המערכת בסיבוכיות מקום וסיבוכיות זמן לכל הפונקציות הנדרשות נממש את המבנה הבא:

ראשית נבנה מחלקות עבור סטודנטים והקורסים ואז באמצעות תבנית של עצי AVL נחזיק עץ עבור הסטודנטים ועץ עבור הקורסים כך שבכל קורס יהיה גם עץ עם הסטודנטים אשר רשומים לקורס (לכל סטודנט נחזיק `shared_ptr`, על מנת לנהל זיכרון נכון ולבדוק כמות מצביעים לסטודנט בעת מחיקה, אם גדול ממש מ-1 לא נמחק כי רשום לקורס).

בנוסף נחזיק משתנה גלובאלי אשר סופר את הסכום המצטבר של הנק"ז אשר דיוקן הפקלוטה העניק לסטודנטים, תחילה יאותחל ל-0 (בהמשך יפורט השימוש במשתנה לחישוב הבונוס לכל סטודנט).

מצורפת תמונה להמחשה:

UML:



אלגוריתמים:

א. Techsystem():

תיאור: נאתחל את המערכת במפורט בסעיף מבנה הנתונים עם עצים ריקים והמשתנה עבור סכום בונוס הנק"ז ל-0.

סיבוכיות: הסיבוכיות הינה $O(1)$ כיוון שמתאחלים מספר סופי של עצים ומשתנה נוסף ולכל אחד הסיבוכיות הינה $O(1)$.

ב. virtual ~techSystem():

תיאור: נשתמש בהורס דיפולטיבי על מנת לקרוא להורס של עצי ה-AVL אשר מבצע מחיקות של צמתים רקורסיבית ומשחרר את כל ה-Nodes. שחרור ה-node גורס להורס shared_ptr שבו, והקורס/סטודנט ישוחררו בסוף כשאר countn יגיע ל-0.

סיבוכיות: הסיבוכיות הינה $O(n*m)$ כיוון שלכל היותר כל ה-n סטודנטים רשומים לכל ה-m קורסים ולכן נמחוק לכל היותר $m*(n+1)$ סטודנטים ו-m קורסים לכן:
$$T(n) = O(n*m) + O(n) + O(m) = O(m*n)$$

ג. StatusType addStudent (int studentId):

תיאור: ניצור node של סטודנט המאותחל עם מפתח studentId וערך shared_ptr לסטודנט אשר מאותחל עם מינוס הבונוס הכולל ונוסיף לעץ הסטודנטים (אם לא קיים שם כבר) לפי גודל המפתח.

סיבוכיות: הסיבוכיות הינה $O(\log n)$ כיוון שסיבוכיות יצירת ה-noden הינה $O(1)$ והכנסה לעץ הינה $O(\log n)$ כפי שנלמד בכיתה לגבי עצי AVL ולכן
$$T(n) = O(1) + O(\log n) = O(\log n)$$

ד. StatusType removeStudent(int studentId):

תיאור: נחפש את הסטודנט בעץ הסטודנטים, אם קיים ואם כמות הקורסים שלוקח הינה 0 נבצע מחיקה שלו מהעץ.

סיבוכיות: סיבוכיות הזמן הינה $O(\log n)$ כיוון שמציאת סטודנט ומחיקתו אם קיים הינה $O(\log n)$ כפי שנלמד בהרצאה, ובדיקת כמות הקורסים שלו הינה $O(1)$ (באמצעות shared_ptr) ולכן $T(n) = O(\log n) + O(1) = O(\log n)$.

ה. :StatusType addCourse (int courseId, int points)

תיאור: ניצור node של קורס המאוחל עם מפתח courseId וערך shared_ptr לקורס אשר מאוחל עם כמות הנקודות ב-points ונוסיף לעץ הקורסים (אם לא קיים שם כבר) לפי גודל המפתח.

סיבוכיות: הסיבוכיות הינה $O(\log n)$ כיוון שסיבוכיות יצירת הnode הינה $O(1)$ והכנסה לעץ הינה $O(\log n)$ כפי שנלמד בכיתה לגבי עצי AVL ולכן
$$T(n) = O(1) + O(\log n) = O(\log n)$$

ו. :StatusType removeCourse (int courseId)

תיאור: נבצע חיפוש של הקורס בעץ הקורסים, אם לא קיים נצא ואם קיים נבצע בדיקה האם עץ הסטודנטים שלו ריק, אם לא נחזיר שגיאה מתאימה ואם כן נבצע מחיקה.

סיבוכיות: סיבוכיות הזמן הינה $O(\log m)$ כיוון שחיפוש ומחיקה הינם ב- $O(\log m)$ ובדיקה האם עץ הסטודנטים שלוקחים את הקורס הינה $O(1)$ ולכן
$$T(n) = O(\log m) + O(1) = O(\log m)$$

ז. :StatusType enrollStudent (int studentId, int courseId)

תיאור: נבצע חיפוש של הסטודנט בעץ הסטודנטים ושל הקורס בעץ הקורסים, אם אחד מהם לא קיים נחזיר שגיאה מתאימה ואם קיימים נבדוק עבור הקורס בעץ הסטודנטים שלו האם הסטודנט חלק מהקורס, אם הסטודנט חלק מהקורס נחזיר שגיאה מתאימה, ואם לא נכניס אותו לקורס.

סיבוכיות: סיבוכיות הזמן הינה $O(\log n + \log m)$ כיוון שחיפוש הסטודנט הינו $O(\log n)$ וחיפוש הקורס הינו $O(\log m)$, וחיפוש הסטודנט בתוך הקורס הינו $O(\log n_{courseId})$ ולכן גם $O(\log n)$ כיוון ש $n_{courseId} \leq n$ ולכן
$$T(n) = O(\log m) + O(\log n) + O(\log n) = O(\log m + \log n)$$

ח. **:StatusType completeCourse (int studentId, int courseId)**

תיאור: נחפש את הקורס בעץ הקורסים, אם הקורס לא קיים נחזיר שגיאה מתאימה ואם קיים נבדוק בעץ הסטודנטים של הקורס האם הסטודנט רשום לקורס, אם לא רשום נחזיר שגיאה ואם רשום נוציא אותו מהקורס ונעלה עבורו את הנק"ז באמצעות shared_ptr שלו (נשים לב כי אין צורך לחפש את הסטודנט בעץ הסטודנטים כיוון שאם הסטודנט לא קיים בכלל ואם קיים אך לא נמצא בקורס דורשות אותה שגיאה בהחזרה ואין לציין במפורש שהסטודנט לא קיים בכלל במערכת).

סיבוכיות: סיבוכיות הזמן הינה $O(\log n_{courseId} + \log m)$ כיוון שחיפוש הקורס הינו $O(\log m)$, סיבוכיות חיפוש והוצאת הסטודנט בעץ הקורס הינה $O(\log n_{courseId})$ ועדכון הנק"ז של הסטודנט הינו $O(1)$ ולכן
$$T(n) = O(1) + O(\log m) + O(\log n_{courseId}) = O(\log n_{courseId} + \log m)$$

ט. **:StatusType awardAcademicPoints (int points)**

תיאור: נעדכן את ערך המשתנה המייצג את סכום נקודות הבונוס שהדיקן העניק עד כה, כך כאשר נרצה לבדוק את כמות הנק"ז של סטודנט מסויים נוכל לחשב את כמות הנק"ז שלו ועוד הבונוס (כי המינוס שאותחל לו ייתקזז עם הבונוס שלא היה אמור לקבל כי נרשם לאחר מכן).

סיבוכיות: סיבוכיות הזמן הינה $O(1)$ כיוון שרק מעדכנים משתנה יחיד.

י. **:output t < int > getStudentPoints (int studentId)**

תיאור: נבצע חיפוש של הסטודנט בעץ הסטודנטים, אם לא קיים נחזיר שגיאה מתאימה ואם קיים, נבדוק את כמות הנק"ז שלו (באמצעות הוספת המשתנה שלנו של הבונוס הכללי) ונחזיר הצלחה ואת כמות הנק"ז.

סיבוכיות: סיבוכיות הזמן הינה $O(\log n)$ כיוון שחיפוש סטודנט בעץ הינו $O(\log n)$ ואם קיים החזרת הנק"ז הינה $O(1)$ לכן $T(n) = O(1) + O(\log n) = O(\log n)$.